claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_c5395511-084\agent-a64b71f3f2cfda1ad.jsonl`
- SHA-256: `425ae470e641adaa226053a2daaa0a0c4d2e38227dfdc1e735cb893d52e84bdb`
- Source modified: `2026-07-06T21:30:48+00:00`
- Imported at: `2026-07-08T16:00:10+00:00`
- project: `wf_c5395511-084`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T21:29:24.642Z)

```
Reviewing GitHub PR #51 "feat(policy): add F2 topic preferences" (branch feat/f2-topic-
preferences -> main). ADR 0012 F2.
+417/-16, 8 files. A checked-out copy is at C:/Users/avidu/Projects/_wt-pr51 (read files there,
or 'git -C C:/Users/avidu/Projects/Telegram show origin/feat/f2-topic-preferences:<path>').
Run repros: PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>  (db._apply_schema on
aiosqlite ":memory:").

WHAT F2 DOES:
- models.py: VideoRecord gains a 'text' field (caption). db.py: insert_video /
_content_item_to_video /
  get_video_by_content_fingerprint / list_videos_for_user propagate text into/out of
content_items.text (which existed since ADR 0011).
- policy.py: TopicPreferences + parse_topic_preferences + load_topic_preferences
(json.loads(policy.config_json)); TopicPreferenceStage
  (Tier 2): exclude-veto, include any/all, casefold substring match over source.title +
content_item.text.
- telethon_client.py: _message_text(message) (message/raw_text, strip, None if empty);
_content_item_preview(...) builds a throwaway
  ContentItem (incl. a computed dedup_key) for topic eval; _evaluate_topic_preferences(conn,
account, source, item) loads the default
  policy and runs TopicPreferenceStage. on_new_message (and backfill.py) now:
matched=engine.evaluate(); if matched and source:
  run topic stage, matched=topic_result.passed; then insert VideoRecord(text=...) and deliver if
matched.
- The live handler on_new_message and backfill_user are '# pragma: no cover'.

VERIFIED by the main reviewer already: caption text reaches content_items.text; topics survive a
/set (F1's merge) and are read via
the RAW config (not FilterConfig); topic gating passes all-includes / vetoes excludes. Gate:
ruff clean, 513 passed @ 95.58%.
Your job: find bugs/regressions/edge cases in the WIRING and stage the happy path missed.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR's diff. No
praise. Concrete failure scenario.
Rank most-severe first. Severity: high (real bug/regression/wrong delivery decision), medium
(edge/latent/design-inconsistency that will bite), low (hygiene/scope-note).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read cited code, run a repro with
PYTHONPATH=C:/Users/avidu/Projects/_wt-pr51/src
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe.
CONFIRMED only if the defect genuinely ships in THIS PR's diff and produces the described wrong
behaviour. REFUTED if handled / can't occur /
```

pre-existing / out-of-scope-and-disclosed. PLAUSIBLE if unsettled. Corrected severity.
FINDING (stage_config): Config error handling is inconsistent: unknown keys fail open (silently disable filtering) while invalid values fail closed (silently drop all)
severity medium | src/tg_video_filter/policy.py:77
summary: A typo in a topics key (e.g. 'includes' instead of 'include') is silently ignored and treated as 'topics disabled' ? everything passes ? whereas a typo in a value (bad mode / non-array include) drops everything; the two config-mistake classes produce opposite delivery outcomes with no signal for either.
scenario: Operator intends to include only 'sports' but writes {"topics": {"includes": ["sports"]}} (extra 's'). parse_topic_preferences only reads the exact keys 'include'/'exclude'/'mode' via config.get; the unknown 'includes' key is ignored, prefs.enabled is False, and the stage passes ALL videos (topic filtering silently off) ? the opposite of the operator's intent. Repro confirmed (case B): `typo key 'includes' -> treated as disabled -> pass: True - topic preferences disabled`. Meanwhile a bad value (finding 1/2) drops all. So two similar config typos yield opposite, silent behaviors, both undetectable without reading logs.
evidence: policy.py:79-89 reads only known keys via raw_topics.get(...) with no rejection of unknown keys; enabled is bool(include or exclude) (policy.py:73-74). Repro r2.py case B output above. Contrast with fail-closed findings 1 and 2.
rec: Pick one consistent posture. If topics config is meant to be strict, reject unknown keys in parse_topic_preferences (raise on any key not in {include, exclude, mode}) so a typo surfaces the same way a bad value does ? ideally validated at write time in save_policy_config. If lenient, both classes should fail open to 'disabled'. The current split (unknown key = open, bad value = closed) is the trap.

2. assistant (2026-07-06T21:29:27.626Z)

I'll adversarially verify this finding about inconsistent config error handling.

3. user (2026-07-06T21:29:28.280Z)

```
1       """Policy pipeline primitives for ADR 0012.
2
3       The policy layer is deliberately small and framework-free in F1: stages receive
4       an immutable context, return a structured result, and the evaluator stops on
5       the first failure. Later Track F phases add concrete stages.
6       """
7
8       from __future__ import annotations
9
10      import json
11      from dataclasses import dataclass, field
12      from typing import Protocol
13
14      from tg_video_filter.models import ContentItem, Policy, Source
15
16
17      @dataclass(frozen=True)
18      class PolicyContext:
19          """Immutable snapshot available to a policy stage."""
20
21          content_item: ContentItem
22          source: Source
23          policy: Policy
24          fetcher: object | None = None
25          budget: object | None = None
26
27
28      @dataclass(frozen=True)
29      class StageResult:
30          """Outcome of one policy stage."""
31
32          passed: bool
33          reason: str
34          stage_name: str
35          metadata: dict[str, object] = field(default_factory=dict)
```

```

        @staticmethod
        def pass_(stage_name: str, reason: str, **metadata: object) -> StageResult:
            return StageResult(
                passed=True,
                reason=reason,
                stage_name=stage_name,
                metadata=metadata,
            )

        @staticmethod
        def fail(stage_name: str, reason: str, **metadata: object) -> StageResult:
            return StageResult(
                passed=False,
                reason=reason,
                stage_name=stage_name,
                metadata=metadata,
            )


    class PolicyStage(Protocol):
        """A single stateless gate in the policy pipeline."""

        async def evaluate(self, ctx: PolicyContext) -> StageResult:
            """Evaluate ``ctx`` and return a stage result."""
            ...


    @dataclass(frozen=True)
    class TopicPreferences:
        """Parsed ADR 0012 topic preference config."""

        include: tuple[str, ...] = ()
        exclude: tuple[str, ...] = ()
        mode: str = "any"

        @property
        def enabled(self) -> bool:
            return bool(self.include or self.exclude)


    def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
        """Parse ``topics`` from a raw policy config object."""
        raw_topics = config.get("topics")
        if raw_topics is None:
            return TopicPreferences()
        if not isinstance(raw_topics, dict):
            raise ValueError("topics must be a JSON object")
        mode = raw_topics.get("mode", "any")
        if mode not in ("any", "all"):
            raise ValueError("topics.mode must be 'any' or 'all'")
        return TopicPreferences(
            include=_string_tuple(raw_topics.get("include")),
            exclude=_string_tuple(raw_topics.get("exclude")),
            mode=str(mode),
        )


    def load_topic_preferences(policy: Policy) -> TopicPreferences:
        """Load topic preferences from ``policy.config_json``."""
        decoded = json.loads(policy.config_json)
        if not isinstance(decoded, dict):
            raise ValueError("policy config_json must decode to a JSON object")
        return parse_topic_preferences(decoded)

```

```
101
102     def _string_tuple(value: object) -> tuple[str, ...]:
103         if value is None:
104             return ()
105         if not isinstance(value, list):
106             raise ValueError("topic include/exclude values must be arrays")
107         topics: list[str] = []
108         for item in value:
109             if not isinstance(item, str):
110                 raise ValueError("topic include/exclude arrays must contain strings")
111             topic = item.strip()
112             if topic:
113                 topics.append(topic)
114         return tuple(topics)
115
116
117     class TopicPreferenceStage:
118         """ADR 0012 Tier 2 topic matching over source title and captured caption."""
119
120         stage_name = "topic-preferences"
121
122         async def evaluate(self, ctx: PolicyContext) -> StageResult:
123             try:
124                 prefs = load_topic_preferences(ctx.policy)
125             except (json.JSONDecodeError, ValueError) as exc:
126                 return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
127             if not prefs.enabled:
128                 return StageResult.pass_(self.stage_name, "topic preferences disabled")
129
130             haystack = _topic_haystack(ctx)
131             matched_excludes = _matched_topics(prefs.exclude, haystack)
132             if matched_excludes:
133                 return StageResult.fail(
134                     self.stage_name,
135                     f"excluded topic matched: {matched_excludes[0]}",
136                     matched_exclude_topics=list(matched_excludes),
137                 )
138
139             if not prefs.include:
140                 return StageResult.pass_(self.stage_name, "no include topics configured")
141
142             matched_includes = _matched_topics(prefs.include, haystack)
143             if prefs.mode == "any":
144                 if matched_includes:
145                     return StageResult.pass_(
146                         self.stage_name,
147                         f"matched include topic: {matched_includes[0]}",
148                         matched_include_topics=list(matched_includes),
149                     )
150                 return StageResult.fail(self.stage_name, "no include topics matched")
151
152             missing = tuple(topic for topic in prefs.include if topic not in
matched_includes)
153             if missing:
154                 return StageResult.fail(
155                     self.stage_name,
156                     f"missing required topic: {missing[0]}",
157                     matched_include_topics=list(matched_includes),
158                     missing_include_topics=list(missing),
159                 )
160             return StageResult.pass_(
161                 self.stage_name,
162                 "matched all include topics",
163                 matched_include_topics=list(matched_includes),
164             )
```

```
165
166
167     def _topic_haystack(ctx: PolicyContext) -> str:
168         parts = [ctx.source.title or "", ctx.content_item.text or ""]
169         return "\n".join(parts).casefold()
170
171
172     def _matched_topics(topics: tuple[str, ...], haystack: str) -> tuple[str, ...]:
173         return tuple(topic for topic in topics if topic.casefold() in haystack)
174
175
176     async def evaluate_pipeline(
177         ctx: PolicyContext,
178         stages: list[PolicyStage],
179     ) -> tuple[bool, list[StageResult]]:
180         """Evaluate stages in order and short-circuit on the first failure."""
181         results: list[StageResult] = []
182         for stage in stages:
183             result = await stage.evaluate(ctx)
184             results.append(result)
185             if not result.passed:
186                 return False, results
187         return True, results
188
189
190     __all__ = [
191         "PolicyContext",
192         "PolicyStage",
193         "StageResult",
194         "TopicPreferenceStage",
195         "TopicPreferences",
196         "evaluate_pipeline",
197         "load_topic_preferences",
198         "parse_topic_preferences",
199     ]
200
```

4. assistant (2026-07-06T21:29:31.946Z)

The finding claims two things: (1) unknown keys are silently ignored (fail open ? pass all), and (2) invalid values fail closed (drop all). Let me verify both classes and confirm they produce opposite outcomes. Let me write a repro.

5. user (2026-07-06T21:29:43.164Z)

```
Exit code 1
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r2.py", line 35, in <module>
    asyncio.run(main())
    ~~~~~~~~~~~^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
           ~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r2.py", line 27, in main
```

```
    await run("A correct include", {"topics": {"include": ["sports"]}})
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r2.py", line 16, in run
    ctx = mk_ctx(config)
  File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\r2.py", line 10, in mk_ctx
    source = Source(id=1, account_id=1, title=title)
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 1 validation error for Source
provider_source_id
  Field required [type=missing, input_value={'id': 1, 'account_id': 1, 'title': 'Sports News'},
input_type=dict]
    For further information visit https://errors.pydantic.dev/2.13/v/missing
```

6. assistant (2026-07-06T21:29:45.364Z)

Let me check the model signatures.

7. user (2026-07-06T21:29:47.340Z)

```
123:class Source(BaseModel):
124-    """A configured place we ingest content from (ADR 0011).
125-
126-    Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127-    providers add their own source kinds). ``provider_source_id`` is the
128-    provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129-    """
130-
131-    model_config = ConfigDict(frozen=True)
132-
133-    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134-    account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135-    provider: str = Field(default="telegram", description="Provider namespace")
136-    provider_source_id: str = Field(
137-        description="Provider-native source id (Telegram channel_id, ?)"
138-    )
--
149:class ContentItem(BaseModel):
150-    """A normalized content record discovered from any :class:`Source` (ADR 0011).
151-
152-    Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153-    (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154-    NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155-    ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156-    wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157-    """
158-
159-    model_config = ConfigDict(frozen=True)
160-
161-    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162-    account_id: int = Field(description="FK ? accounts.id")
163-    source_id: int = Field(description="FK ? sources.id")
164-    provider: str = Field(default="telegram", description="Provider namespace")
--
186:class Policy(BaseModel):
187-    """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188-
189-    Generalizes the legacy per-account ``filter_config`` row so one account can
190-    own multiple policies (enables ADR 0009's per-route filter overrides) and so
191-    future enrichment/security stages grow into the same ``config_json`` column.
192-    Exactly one policy per account carries ``is_default=True``; the legacy
193-    ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194-    """
```

```
195-
196-    model_config = ConfigDict(frozen=True)
197-
198-    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199-    account_id: int = Field(description="FK ? accounts.id")
200-    name: str = Field(description="Policy name; 'default' for the account default policy")
201-    config_json: str = Field(
```

8. user (2026-07-06T21:29:51.546Z)

```
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155         ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156         wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157         """
158
159         model_config = ConfigDict(frozen=True)
160
161         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162         account_id: int = Field(description="FK ? accounts.id")
163         source_id: int = Field(description="FK ? sources.id")
164         provider: str = Field(default="telegram", description="Provider namespace")
165         provider_message_id: str = Field(
166             description="Provider-native message id (Telegram message_id, ?)"
167         )
168         content_type: str = Field(default="video", description="'video' | 'link' |
'article'")
169         text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170         link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171         media_ref: str | None = Field(default=None, description="Provider media reference")
172         document_id: str | None = Field(
173             default=None,
174             description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175         )
176         duration_s: int | None = None
177         width: int | None = None
178         height: int | None = None
179         size_bytes: int | None = None
180         mime_type: str | None = None
181         dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182         matched: bool = False
183         seen_at: datetime = Field(default_factory=_utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
```

9. assistant (2026-07-06T21:29:54.500Z)

Now let me fix the repro with required fields. Actually, the finding only needs `parse_topic_preferences` and stage behavior ? the stage only reads `ctx.source.title` and `ctx.content_item.text`. Let me fix the constructors.

10. user (2026-07-06T21:30:05.317Z)

```
A correct include: parsed enabled=True include=('sports',) exclude=()
    stage passed=True reason='matched include topic: sports'
B typo key 'includes': parsed enabled=False include=() exclude=()
    stage passed=True reason='topic preferences disabled'
C bad mode: parse raised ValueError: topics.mode must be 'any' or 'all'
    stage passed=False reason="invalid topic config: topics.mode must be 'any' or 'all'"
D non-array include: parse raised ValueError: topic include/exclude values must be arrays
    stage passed=False reason='invalid topic config: topic include/exclude values must be
arrays'
E typo key, non-matching topic: parsed enabled=False include=() exclude=()
    stage passed=True reason='topic preferences disabled'
```

11. assistant (2026-07-06T21:30:15.121Z)

The repro confirms the factual claims:

- **Case B** (typo key `includes`): parsed `enabled=False`, stage `passed=True` ("topic preferences disabled") ? filtering silently off, everything passes.
- **Case C** (bad `mode` value): raises ValueError, stage `passed=False` ? everything dropped.
- **Case D** (non-array include): raises ValueError, stage `passed=False` ? everything dropped.
- **Case E** confirms the "passes all" is real even for a topic that would NOT match the haystack ? the veto/include logic is bypassed entirely because `enabled=False`.

So the two config-mistake classes (unknown-key vs bad-value) genuinely produce **opposite** silent delivery outcomes. The behavior ships exactly as described.

Now the critical question for verdict: **is this a defect in THIS PR's diff, or an accepted design?** The finding itself categorizes it as `medium` (design-inconsistency that will bite). Let me assess whether this rises to a real defect versus a disclosed/acceptable design choice. Key considerations:

1. The fail-open-on-unknown-key is inherent to `dict.get()` on exact keys ? a common and arguably acceptable "lenient config" posture. Unknown-key rejection is genuinely uncommon in JSON config parsers.
2. The inconsistency is real: bad-value fails closed while unknown-key fails open. That is the substantive point.
3. Neither behavior is documented/tested as intended in a way I've seen, but the config write path could validate. Let me check whether there's a write-time validation path the finding references (`save_policy_config`).

12. user (2026-07-06T21:30:16.769Z)

```
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:203:async def _apply_schema(conn:
aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:319:async def _apply_migrations(conn:
aiosqlite.Connection) -> None:
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1326:async def save_policy_config(
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1793:    await
save_policy_config(conn, user_id, raw_config)
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\db.py:1862:    "save_policy_config",
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\policy.py:77:def
parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\policy.py:99:    return
parse_topic_preferences(decoded)
C:\Users\avidu\Projects\_wt-pr51\src\tg_video_filter\policy.py:198:
"parse_topic_preferences",
```

13. assistant (2026-07-06T21:30:20.178Z)

`save_policy_config` and `parse_topic_preferences` are not in this PR's diff header set? Let me check whether these are actually part of PR #51's changed lines. The finding is anchored at `policy.py:77` which is in `parse_topic_preferences` ? added by this PR. Let me verify `parse_topic_preferences` is new in this PR.

14. user (2026-07-06T21:30:22.332Z)

```
diff --git a/src/tg_video_filter/policy.py b/src/tg_video_filter/policy.py
index e7d0303..395435a 100644
--- a/src/tg_video_filter/policy.py
+++ b/src/tg_video_filter/policy.py
@@ -7,6 +7,7 @@ the first failure. Later Track F phases add concrete stages.

 from __future__ import annotations

+import json
 from dataclasses import dataclass, field
 from typing import Protocol

@@ -60,6 +61,118 @@ class PolicyStage(Protocol):
         ...


+@dataclass(frozen=True)
+class TopicPreferences:
+    """Parsed ADR 0012 topic preference config."""
+
+    include: tuple[str, ...] = ()
+    exclude: tuple[str, ...] = ()
+    mode: str = "any"
+
+    @property
+    def enabled(self) -> bool:
+        return bool(self.include or self.exclude)
+
+
+def parse_topic_preferences(config: dict[str, object]) -> TopicPreferences:
+    """Parse ``topics`` from a raw policy config object."""
+    raw_topics = config.get("topics")
+    if raw_topics is None:
+        return TopicPreferences()
+    if not isinstance(raw_topics, dict):
+        raise ValueError("topics must be a JSON object")
+    mode = raw_topics.get("mode", "any")
+    if mode not in ("any", "all"):
+        raise ValueError("topics.mode must be 'any' or 'all'")
+    return TopicPreferences(
+        include=_string_tuple(raw_topics.get("include")),
+        exclude=_string_tuple(raw_topics.get("exclude")),
+        mode=str(mode),
+    )
+
+
+def load_topic_preferences(policy: Policy) -> TopicPreferences:
+    """Load topic preferences from ``policy.config_json``."""
+    decoded = json.loads(policy.config_json)
+    if not isinstance(decoded, dict):
+        raise ValueError("policy config_json must decode to a JSON object")
+    return parse_topic_preferences(decoded)
+
+
+def _string_tuple(value: object) -> tuple[str, ...]:
+    if value is None:
```

```
+        return ()
+    if not isinstance(value, list):
+        raise ValueError("topic include/exclude values must be arrays")
+    topics: list[str] = []
+    for item in value:
+        if not isinstance(item, str):
+            raise ValueError("topic include/exclude arrays must contain strings")
+        topic = item.strip()
+        if topic:
+            topics.append(topic)
+    return tuple(topics)
+
+
+class TopicPreferenceStage:
+    """ADR 0012 Tier 2 topic matching over source title and captured caption."""
+
+    stage_name = "topic-preferences"
+
+    async def evaluate(self, ctx: PolicyContext) -> StageResult:
+        try:
+            prefs = load_topic_preferences(ctx.policy)
+        except (json.JSONDecodeError, ValueError) as exc:
+            return StageResult.fail(self.stage_name, f"invalid topic config: {exc}")
+        if not prefs.enabled:
```

15. assistant (2026-07-06T21:30:35.877Z)

Confirmed: `parse_topic_preferences`, `TopicPreferences`, and the entire fail-open/fail-closed behavior is 100% introduced by this PR. This is squarely in the diff, not pre-existing.

Now the severity assessment. The facts are confirmed and ship in this PR. The question is whether "medium" is right:

Arguments for the defect being real and worth flagging:
- The inconsistency is genuine and confirmed: a typo in a key silently disables all filtering (over-delivery), while a typo in a value silently drops everything (under-delivery). Both are silent.
- Over-delivery (fail-open on typo key) is arguably the worse of the two for a *filtering* system whose purpose is to exclude unwanted content ? a veto typo like `{"topics": {"exclude": ...}}` written as `"excludes"` would let excluded content through silently.

Arguments for lower / not-a-bug:
- Fail-open on unknown keys is standard JSON-config behavior (`dict.get`). Most config parsers don't reject unknown keys. Calling this a "bug" is a stretch; it's a design-consistency nit.
- The value-typo fail-closed is defensible defensive behavior (a malformed value is a hard error). The asymmetry is a design smell, not a functional bug in a code path with a single intended input.
- No wrong behavior occurs for *valid* config. This only manifests under operator typos, and there's no evidence the operator-facing write path (`save_policy_config`) is expected to validate here.

This is a legitimate design-inconsistency observation, correctly self-classified as medium (an edge/latent design-inconsistency that will bite). It is not a high-severity functional bug ? no valid input misbehaves. But it's also not refuted: the asymmetric silent behavior genuinely ships. I'll confirm at medium.

16. user (2026-07-06T21:30:48.557Z)

Structured output provided successfully